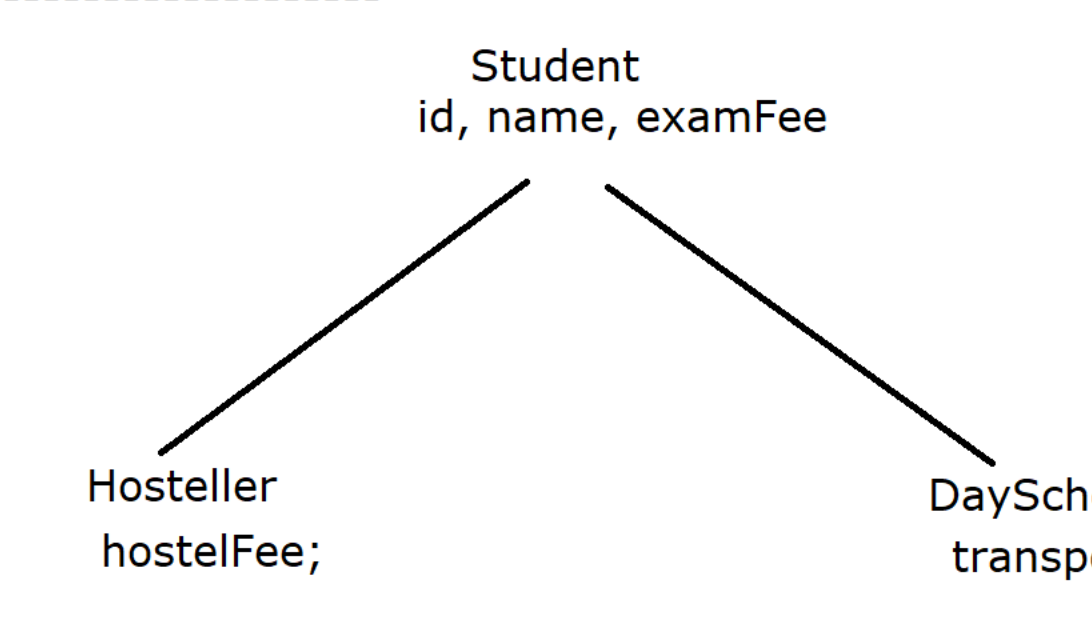
//Program on Hierarchical Inheritance :



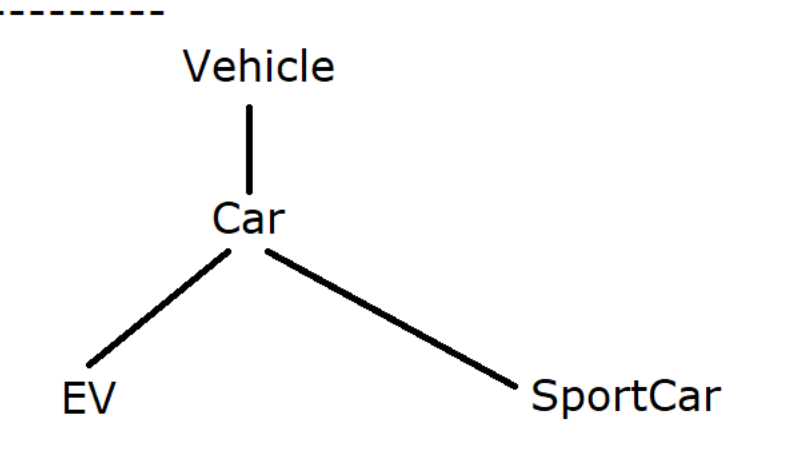

```java
package com.ravi.hierarchical;

class Student
{
    protected int id;
    protected String name;
    protected double examFee;

    public Student(int id, String name, double examFee) {
        super();
        this.id = id;
        this.name = name;
        this.examFee = examFee;
    }
}

class Hosteller extends Student
{
    protected double hostelFee;

    public Hosteller(int id, String name, double examFee, double hostelFee)
    {
        super(id, name, examFee);
        this.hostelFee = hostelFee;
    }

    @Override
    public String toString()
    {
        return "Hosteller [id=" + id + ", name=" + name + ", examFee=" + examFee + ", hostelFee="
+ hostelFee + "]";
    }
}

class DayScholer extends Student
{
    protected double transportFee;

    public DayScholer(int id, String name, double examFee, double transportFee)
    {
        super(id, name, examFee);
        this.transportFee = transportFee;
    }

    @Override
    public String toString()
    {
        return "DayScholer [id=" + id + ", name=" + name + ", examFee=" + examFee + ",
transportFee=" + transportFee
                + "]";
    }
}
public class HierarchicalDemo1
{
    public static void main(String[] args)
    {
        Hosteller hosteller = new Hosteller(1, "Scott", 3500, 130000);
        IO.println(hosteller);

        DayScholer dayScholer = new DayScholer(2, "John", 3500, 90000);
        IO.println(dayScholer);
    }
}
```

//Program on multilevel Inheritance :

A -> B -> C

```java
package com.ravi.multilevel_inheritance;

class Vehicle
{
    protected String brand;
    protected int model;

    public Vehicle(String brand, int model)
    {
        super();
        this.brand = brand;
        this.model = model;
    }

    public void displayVehicleDetails()
    {
        IO.println("Brand Name is :"+this.brand);
        IO.println("Model  is :"+this.model);
    }
}

class Car extends Vehicle
{
    protected int noOfDoors;

    public Car(String brand, int model, int noOfDoors)
    {
        super(brand, model);
        this.noOfDoors = noOfDoors;
    }

    public void displayCarDetails()
    {
        IO.println("Number of doors :"+this.noOfDoors);
    }
}

class SportCar extends Car
{
    protected int topSpeed;

    public SportCar(String brand, int model, int noOfDoors, int topSpeed)
    {
        super(brand, model, noOfDoors);
        this.topSpeed = topSpeed;
    }

    public void displaySportCar()
    {
        IO.println("Top speed is :"+this.topSpeed);
    }
}

public class MultilevelInheritance {

    public static void main(String[] args)
    {
        SportCar sportCar = new SportCar("Ferrari", 2026, 2, 450);
        sportCar.displayVehicleDetails();
        sportCar.displayCarDetails();
        sportCar.displaySportCar();

        IO.println(".................");
        Car car = new Car("Maruti", 2026, 4);
        car.displayVehicleDetails();
        car.displayCarDetails();
    }

}
```

Assignment on Hybrid Inheritance :



***** Why Java does not support Multiple Inheritance ?**

```
class A                          class B
{                                {
   A()        Diamond Problem       B()
   {                                {
   }                                }
}                                }
          class C extends A,B  //Invalid
          {
             C()
             {
               super(); //Ambiguity issue
             }
          }
```

Multiple Inheritance is a situation where a sub class wants to inherit the properties two or more than two super classes.

In every constructor we have super() or this() to the first line. When compiler will add super() to the first line of the constructor then we have an ambiguity issue that super() will call which super class constructor as shown in the diagram [either A class OR B class]

It is also known as Diamond Problem in java so the final conclusion is we can't achieve multiple inheritance using classes but same we can achieve by using interface [interface does not contain any constructor]

JVM Architecture with class loader sub system :
----

* The entire JVM Architecture is divided into 3 parts :

1) Class Loader subsystem
2) Runtime Data Areas (Memory Areas)
3) Execution Engine [Interpreter + JIT Compiler]

1) Class Loader Subsystem :
----

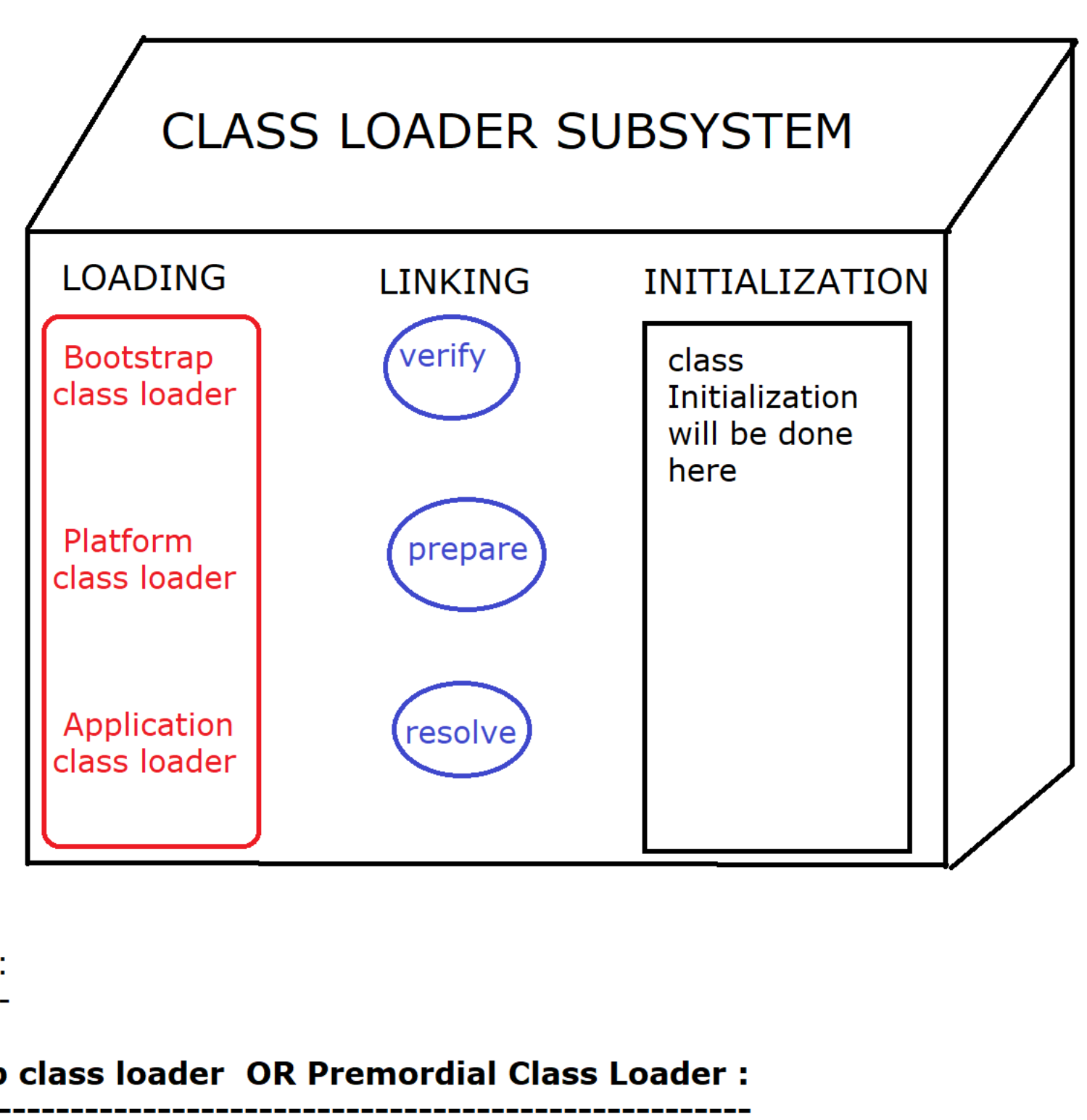
* The main purpose of **class loader subsystem** is to load the .class file into JVM memory.

* Once the .class file will be loaded into JVM memory then only the execution of the program will be started.

* In order to load the .class file into JVM memory, class loader subsystem internally uses an algorithm called "Delegation Hierarchy Algorithm".

* Class Loader subsystem internally performs 3 tasks :

a) LOADING   b) LINKING   c) INITIALIZATION



LOADING :

Bootstrap class loader OR Premordial Class Loader :
